Practical8

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int i;

   printf("=== malloc() ===\n");

   int *a = (int *)malloc(5 * sizeof(int));

   if (a == NULL)
   {
      printf("malloc failed\n");
      return 1;
   }

   for (i = 0; i < 5; i++)
   {
      a[i] = (i + 1) * 10;
   }

   printf("Values allocated using malloc:\n");

   for (i = 0; i < 5; i++)
   {
```

```
    printf("%d ", a[i]);
}

printf("\n");



printf("\n=== calloc() ===\n");

int *b = (int *)calloc(5, sizeof(int));

if (b == NULL)
{
    printf("calloc failed\n");
    free(a);
    return 1;
}

printf("Values allocated using calloc:\n");

for (i = 0; i < 5; i++)
{
    printf("%d ", b[i]);
}

printf("\n");



printf("\n=== realloc() ===\n");
```

```
int *temp = (int *)realloc(a, 10 * sizeof(int));

if (temp == NULL)
{
   printf("realloc failed\n");
   free(a);
   free(b);
   return 1;
}

a = temp;

for (i = 5; i < 10; i++)
{
   a[i] = (i + 1) * 10;
}

printf("Values after realloc:\n");

for (i = 0; i < 10; i++)
{
   printf("%d ", a[i]);
}

printf("\n");
```

```
    printf("\n=== free() ===\n");

    free(a);
    free(b);

    printf("Memory successfully released.\n");

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ 8.  Develop a program using malloc(), ca
lloc(), realloc(), and free(). Monitor memory allocation behavior and identi
fy memory leaks using Valgrind.

Develop a program that demonstrates Copy-on-Write behavior after a fork(). A
nalyze memory usage before and after modifying data in the child process.
^C
snigdha@Snigdha:~/quotes_practical$ nano memory_alloc.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -g -o memory_alloc mem
ory_alloc.c
snigdha@Snigdha:~/quotes_practical$ ./memory_alloc
=== malloc() ===
Values allocated using malloc:
10 20 30 40 50

=== calloc() ===
Values allocated using calloc:
0 0 0 0 0

=== realloc() ===
Values after realloc:
10 20 30 40 50 60 70 80 90 100

=== free() ===
Memory successfully released.
snigdha@Snigdha:~/quotes_practical$
```